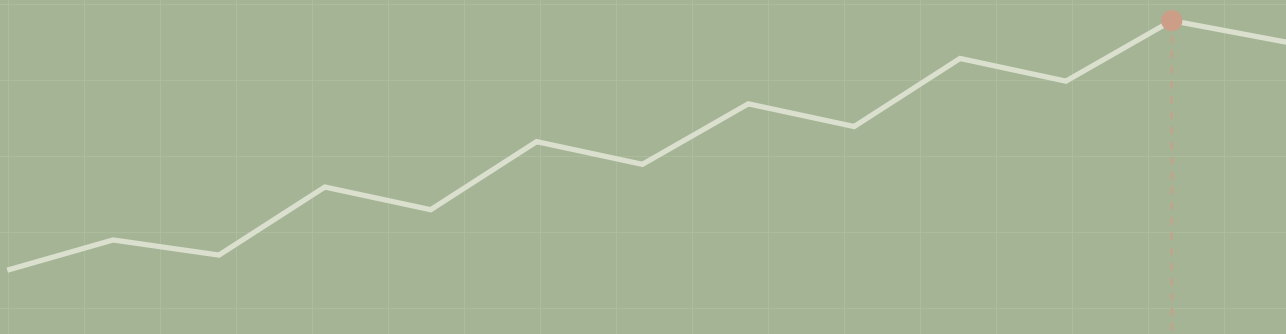


Keepr *Analytics*

La console d'exploitation d'une messagerie familiale privée : voir l'usage, les coûts et les accès sensibles — sans jamais toucher au contenu des conversations, ni apparaître dans l'app.



5

sources de signal,
chacune avec son propre
mode de collecte

2

verrous d'accès côté
serveur — règles de base
et fonctions

0

contenu de message
collecté par la télémétrie

1

concepteur-développeur,
de la spécification au
déploiement

Piloter un service privé *sans regarder dedans.*

Keepr est une PWA de messagerie utilisée par un cercle restreint — famille et proches. Dès qu'un service tourne en production, il faut savoir s'il est utilisé, ce qu'il coûte et qui touche à l'administration. La difficulté : obtenir ces réponses sans créer un outil de surveillance des personnes.

Données personnelles

Les conversations sont privées. L'observabilité doit fonctionner avec des compteurs et des métadonnées, jamais avec du contenu.

Coût quasi nul

Projet personnel sur facturation cloud à l'usage : chaque écriture compte, et aucun pipeline ne doit tourner « au cas où ».

Un seul opérateur

Pas d'astreinte, pas d'équipe. Les panneaux doivent être lisibles d'un coup d'œil et fiables sans maintenance quotidienne.

Risque financier réel

Un mécanisme de coupure budgétaire protège le projet. L'observabilité ne doit jamais pouvoir l'empêcher de fonctionner.

Zéro impact produit

Aucun écran, aucune latence, aucun bug d'analytics ne doit être perceptible par les utilisateurs de l'app.

RÉPONDRE À

Est-ce utilisé ?

Quels espaces de l'app vivent, lesquels sont délaissés, jour après jour.

RÉPONDRE À

Combien ça coûte ?

Où en est la consommation face au budget, et quels services la tirent.

RÉPONDRE À

Qui administre ?

Chaque tentative d'accès aux fonctions d'administration, acceptée ou refusée.

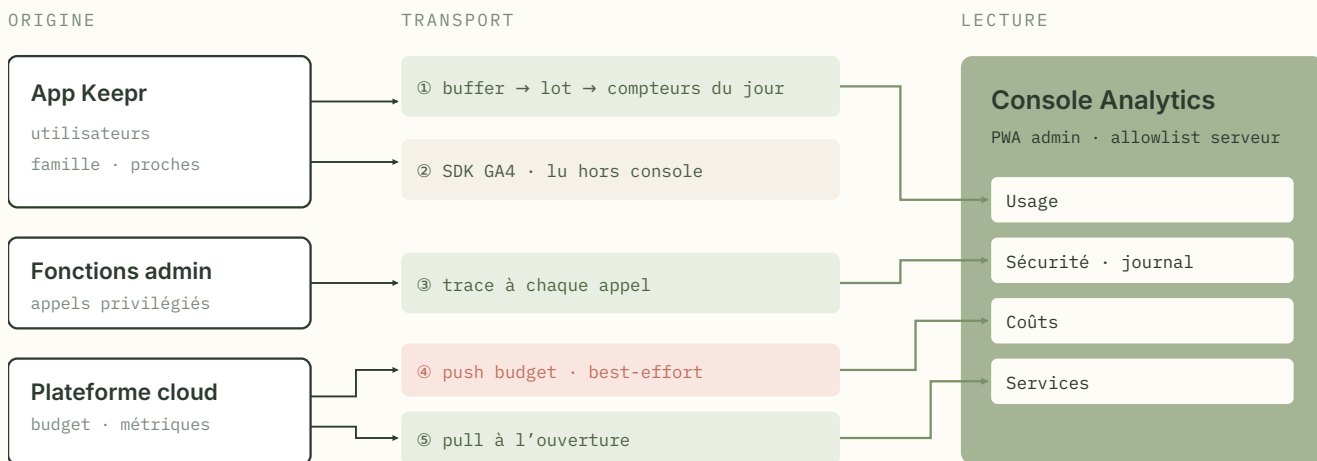
CE QUE LA CONSOLE N'EST PAS

- Un écran caché dans l'app familiale
- Un entrepôt de données parallèle

- Un lecteur de conversations
- Un maillon du mécanisme de coupure

Cinq signaux, *cinq modes de collecte.*

Chaque source est traitée avec le mode qui lui convient — lot, push, pull, externe. Aucune ne sert de pont vers une autre : si l’une tombe, les autres restent lisibles.



1	Télémétrie maison LOT	Événements de navigation mis en tampon côté client, envoyés par lot à une Cloud Function qui incrémente des compteurs journaliers. Aucune ligne brute par geste.	POURQUOI Tendance quotidienne lisible pour une fraction des écritures d'un log événementiel.
2	GA4 en parallèle EXTERNE	Mesure produit tierce, instrumentée au même point que la télémétrie maison mais jamais à sa place. Consultée dans l'interface GA4, pas dans la console.	POURQUOI Un bloqueur, un filtre ou une panne côté tiers ne rend pas l'opérateur aveugle.
3	Journal de sécurité TRACE	Chaque appel aux fonctions d'administration est tracé, qu'il soit accepté ou refusé. S'y ajoute un suivi des constats d'audit de sécurité, par sévérité et statut.	POURQUOI Reconstituer qui a tenté quoi, sans dépendre de la mémoire d'une session.
4	Coûts cloud PUSH	Les notifications de budget qui transitent déjà par le mécanisme de coupure sont interceptées et enregistrées — après la décision de coupure, dans un bloc isolé.	POURQUOI Un signal de consommation suffit à piloter, sans API de facturation ni export d'entrepôt.
5	Usage des services PULL	Métriques de la plateforme lues à la demande, à l'ouverture du panneau. Rien n'est recopié ni mis en cache.	POURQUOI Pas de pipeline permanent, pas de seconde vérité qui dérive de la source.

Chaque choix *a un prix, assumé.*

Les arbitrages sont documentés comme des décisions d'architecture : l'option retenue, celle qui a été écartée, et ce que l'on accepte de perdre en échange.

DÉCISION	ALTERNATIVE ÉCARTÉE	RAISON	COÛT ACCEPTÉ
Deux mesures d'usage indépendantes	Tout dans GA4, ou tout en maison	Résilience : chaque canal peut tomber sans aveugler l'autre.	Double instrumentation ; chiffres non réconciliables entre eux.
Compteurs journaliers agrégés	Log brut d'événements	Écritures et stockage minimaux ; minimisation des données par construction.	Pas d'analyse de parcours individuel — volontairement.
Envoi par lot, sans attente	Une écriture par interaction	Coût maîtrisé ; aucun échec de tracking ne bloque l'interface.	Quelques événements perdus si l'app se ferme avant l'envoi.
Coûts via notifications budget	API de facturation, export entrepôt	Réutilise un flux existant ; zéro infrastructure de plus.	Granularité et fraîcheur limitées à ce que publient les alertes.
Écriture des coûts hors chemin critique	Traçage intégré à la logique de coupure	La protection financière prime sur la visibilité.	Un relevé peut manquer ; la coupure, jamais.
Métriques services en pull	Collecte planifiée et stockée	Donnée toujours à jour, aucun job permanent.	Un appel à chaque ouverture ; historique limité à la rétention de la source.
Console = PWA distincte	Écran caché dans l'app	Aucune surface admin exposée ; bundle de l'app non alourdi.	Deux applications installables à faire cohabiter sur un même hébergement.
Accès vérifié côté serveur, deux fois	Masquage de l'écran côté client	Un client modifié ne suffit pas à lire ni à écrire.	Allowlist à maintenir à deux endroits, de façon cohérente.

PRINCIPE DIRECTEUR

L'observabilité ne doit jamais coûter plus cher, ni être plus fragile, que le service qu'elle observe.

La confiance ne vient *jamais du navigateur.*

L'accès à la console n'est pas un écran masqué : c'est un contrôle serveur, appliqué deux fois, et tracé. Le frontend affiche ; il ne décide rien.

CLIENT	VERROU 1	VERROU 2	TRACE
Interface console Peut être modifiée par quiconque. Considérée comme non fiable par principe.	Règles de base Les collections d'analytics refusent toute lecture et écriture hors allowlist.	Cloud Functions Chaque fonction d'administration revérifie l'identité avant d'agir.	Journal Accepté ou refusé, chaque appel laisse une trace consultable.

MINIMISATION – CE QUE LA TÉLÉMÉTRIE CONTIENT

COLLECTÉ
Nom d'espace de l'app consulté
Compteur agrégé par jour
Montants et seuils de budget
Métriques de fonctionnement des services
Horodatage et issue des appels admin

JAMAIS COLLECTÉ
Texte des messages
Pièces jointes, médias, transcriptions
Parcours individuel par utilisateur
Copie des conversations, même partielle
Secrets ou jetons dans les logs

Fire-and-forget

Un échec d'envoi de télémétrie est absorbé silencieusement et localement. L'utilisateur ne le voit jamais.

Hébergement européen

Stack hébergée en région Europe.
Pas d'entrepôt secondaire hors de la plateforme.

Surface publique minimale

Identifiants, comptes de service et seuils opérationnels ne figurent dans aucun document public.

Livrer seul, *avec des garde-fous d'équipe.*

Le code est produit avec un agent IA (Claude Code), piloté comme une équipe : consignes cadrées, rapports contredits, preuves exigées. La rigueur vient du processus, pas de la confiance.

01	02	03	04 · PORTE
Reconnaissance	Consigne	Staging	Validation
Lecture du code source avant toute écriture. Pas de correctif fondé sur une hypothèse.	Périmètre, fichiers interdits et critères d'acceptation écrits avant l'exécution.	Déploiement automatique, tests e2e en CI, contrôle du build réellement servi.	Mise en production uniquement après validation manuelle sur appareil.

Règles avant interface	Les règles d'accès sont déployées avant le frontend qui en dépend — jamais l'inverse.
Fichiers sensibles nommés	Chaque diff est vérifié contre une liste explicite de fichiers intouchables : règles, config d'hébergement, pipelines CI, logique de coupure.
Isolation du bundle	La console n'importe aucun composant partagé qui alourdirait son bundle ; les éléments sont réimplémentés localement.
Une session agent à la fois	Le dépôt est partagé avec le produit principal : pas d'exécutions concurrentes.
Incidents consignés	Pièges connus et incidents sont documentés dans le dépôt, avec leur cause et la règle qui en découle.

STACK

- Firebase Hosting, Cloud Functions (Node 22), Firestore
- Pub/Sub, Cloud Monitoring, Secret Manager
- React, Vite · PWA installable
- GitHub Actions · Playwright e2e

CHANTIERS OUVERTS

- Suivi des exécutions CI dans la console
- Détail par Cloud Function sur 24 h et 7 jours
- Tests unitaires des états d'erreur des panneaux
- Protection de branche sur la production

CONCEPTION & RÉALISATION

Mathieu El Almawie

Business Analyst MOA · PO · conception produit et architecture de Keepr

Site mthlabs.github.io/Portofolio
Contact mathieu.elalmawie@gmail.com